Lecture 02: Problem Definition and the AI/ML Lifecycle

NT120 – Introduction to AI for Computer Networks and Cybersecurity

Huynh Van Dang, Phan The Duy, Tran Tuan Dung, Nguyen Khanh Thuat

Faculty of Computer Networks and Communications
University of Information Technology, VNU–HCM

danghv@uit.edu.vn

Course page: link.uit.edu.vn/NT120

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Learning Outcomes

By the end of this lecture, students will be able to:

- Describe the six stages of the AI/ML lifecycle and the feedback loops between them
- Translate an operational security requirement into a well-posed learning task
- Choose an evaluation metric from the cost of each error type, before training
- Explain why security data often requires a temporal rather than a random split
- Identify where human oversight must remain in the loop

Course learning outcomes addressed: G1.2

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Recap of Session 1

- Machine learning *induces* the rule from data instead of having a person write it
- The paradigm follows the data you can obtain; in security, labels are scarce
- Accuracy without a base rate is not information

Today's move. From "which algorithm should I use?" to "which problem am I actually solving?"

- The lifecycle introduced today is the backbone of the whole course
- Every later session will be introduced by naming the stage it belongs to

A model trained on a badly framed problem is not a partial success. It is a system that produces confident answers to a question nobody asked.

Three Ways to Fail Before Training Anything

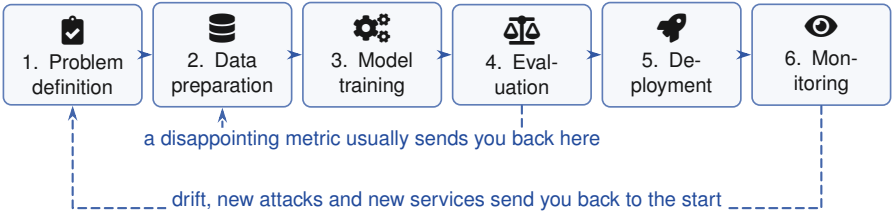
- **The prediction unit is never fixed.** Is a detection made per packet, per flow, per session, per host, or per hour? The same dataset supports all five and they are different problems with different labels and different costs.
- **The label means something other than the decision.** “Traffic to a blocklisted IP” is a convenient label. “An intrusion the analyst would want to see” is the decision. They overlap; they are not the same thing.
- **The metric is chosen after the results.** If you pick the metric once you know the numbers, you have not evaluated the system, you have advertised it.

⚠ Each of these is invisible in the code and fatal in production. None of them is fixed by a better model.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

The Lifecycle Is a Cycle



The dashed arrows are what separate an ML project from a linear software project. A model is never finished; it is only current.

What Happens at Each Stage

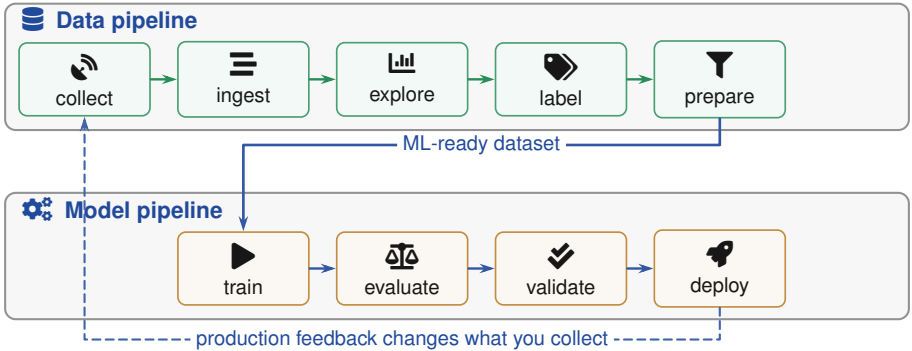
Stage	You decide	You can get it wrong by
1. Problem definition	The decision, the prediction unit, the label, the metric	Framing a question the data cannot answer
2. Data preparation	Sources, cleaning, encoding, the split	Leaking the label into the features
3. Model training	Baseline first, complexity only if justified	Tuning against the test set
4. Evaluation	Whether it meets the criterion fixed in stage 1	Changing the criterion to fit the result
5. Deployment	Latency, throughput, where it sits in the path	Ignoring the cost of an alert on the team
6. Monitoring	Drift alarms, retraining triggers, rollback	Assuming yesterday's accuracy still holds

Sessions 3–5 are stage 2. Sessions 6 and 10 are stage 3. Sessions 7 and 9 are stage 4. Stages 5 and 6 are treated at orientation level in Sessions 13 and 14.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Data Pipeline and Model Pipeline



Data quality caps model quality. No architecture repairs a broken label, and no amount of tuning recovers information the pipeline discarded.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

The Framing Canvas: Six Questions

1. Decision
What action does the output drive, and who takes it?

2. Prediction unit
One flow, session, e-mail, host, or one hour of traffic?

3. Label
What exactly is positive, and which process produced it?

4. Features
What is known *at the moment of prediction*, not afterwards?

5. Horizon
Detect what is happening now, or predict the next hour?

6. Error profile
What does each kind of mistake cost, in operational units?

Fill this in before opening a notebook. It is the deliverable of today's in-class activity and of your homework, and it comes back in Sessions 11 and 15.

Worked Example: “Reduce Phishing Incidents”

Decision	Quarantine, warn the user, or deliver an incoming message
Prediction unit	One e-mail, scored at delivery time
Label	Confirmed phishing after analyst adjudication. *Not* user reports alone: users over-report newsletters and under-report good forgeries
Features	Header fields, sender reputation, URL structure, attachment type, body features. *Not* whether anyone clicked: that is known only afterwards
Horizon	Decide within the delivery path, so within a few hundred milliseconds
Error profile	A missed phish can start an incident; a false quarantine blocks business mail and erodes trust in the filter. Aim for high recall at a precision the helpdesk can absorb

Notice that four of the six rows are decisions about the *problem*, not about the model. Only the last one begins to constrain the algorithm.

The Same Requirement, Framed Two Ways

✗ Framing A

"Detect malicious hosts on the campus network."

- Unit: a host. Over what window?
- Label: what makes a host malicious, and who decides?
- Features: aggregate of everything the host did. Which part?
- Nobody can say what a false positive costs

✓✓ Framing B

"Flag hosts whose outbound DNS behaviour in a one-hour window resembles known DGA activity, for analyst review."

- Unit: one host-hour
- Label: hours containing DNS queries to confirmed DGA domains
- Features: query volume, NXDOMAIN rate, name entropy, unique second-level domains
- Cost: 10 minutes of analyst time per flag

💡 Framing B is narrower, and that is precisely why it can be built, measured and argued about. Narrowing the question is the work.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

The Metric Follows the Cost

Step 1 — write both costs in operational units.

- **False negative** (missed intrusion): incident response hours, dwell time, data loss, regulatory exposure
- **False positive** (false alarm): analyst minutes, alert fatigue, and eventually a team that ignores the tool

Step 2 — let the ratio choose the emphasis.

- Missed intrusion far more costly $\rightarrow$ weight recall, accept lower precision, plan for the alert volume
- Analyst capacity is the binding constraint $\rightarrow$ weight precision, accept that some attacks are missed

Step 3 — write it down and freeze it before any model is trained. Record the primary metric, the threshold policy, and the minimum acceptable value.

You will not be able to compute these costs exactly. Writing down even a rough ratio is far better than leaving it implicit, because it makes the trade-off arguable.

A Preview of Session 7

- Every metric in this course is a summary of the same four numbers: true and false positives, true and false negatives
- **Accuracy** answers “how often am I right?” and is useless when one class is 0.4% of the data
- **Precision** answers “of the alerts I raised, how many were real?” — the analyst’s question
- **Recall** answers “of the real attacks, how many did I catch?” — the risk owner’s question
- The threshold moves you along the trade-off between the last two. It is an operational policy, not a modelling detail

For today it is enough to know that the choice exists and belongs in stage 1.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Train, Validation and Test

- **Training set** fits the parameters
- **Validation set** selects between models and settings
- **Test set** is touched *once*, at the end, to report a number



a typical split; the proportions matter less than the discipline

Any decision informed by the test set contaminates it. Once you have looked, it is a second validation set. Fix and record the

random seed, so that a different result means a different method rather than a different day.

Why Random Splitting Lies on Network Data

Random split



Flows from one capture minute end up on both sides. Near-duplicate flows of the same attack burst appear in train and test. The reported score is optimistic and unrepeatable in production.

Temporal split



Train on the past, test on the future — which is what deployment actually does. Group splitting by capture or by host is the equivalent fix when time is not the confound.

Which Split, When

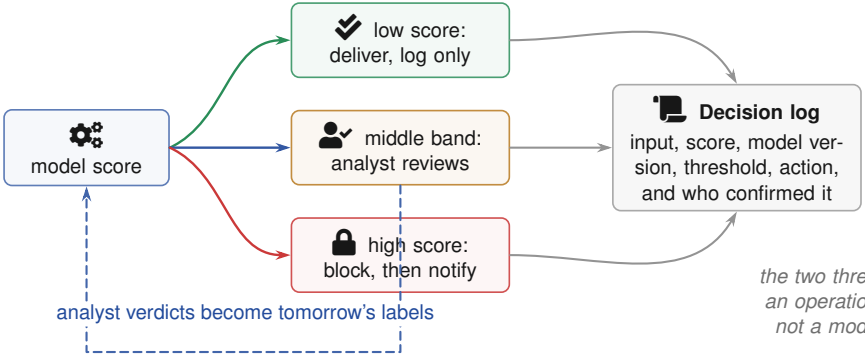
Split strategy	Use when	Guards against
Random	Rows are genuinely independent and order carries no meaning	Nothing in particular; the default, and rarely right here
Stratified random	Classes are imbalanced and rows are independent	A fold with no positive examples at all
Temporal	The system will be deployed forward in time	Leaking future information into the past
Grouped	Rows share a capture, a host, a user or a campaign	Near-duplicates straddling the split

In this course every report must state which split was used and why. “Random, 80/20” on flow data is a claim you must defend, not a default you may assume.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Where the Human Stays in the Loop



Three Rules for Oversight

- **Irreversible actions need a person.** Blocking a production host, disabling an account, or dropping traffic at a border are actions where a false positive has its own incident cost.
- **Log enough to reconstruct the decision.** Input, score, model version, threshold and action. Six months later someone will ask why, and “the model said so” is not an answer.
- **Analyst verdicts are data.** The review queue is your cheapest source of fresh labels — and, as Session 14 shows, a path an attacker can try to poison.

⚠ Accountability does not transfer to a model. If the system acts, a named person owns the outcome. Design the escalation policy on that basis.

Outline

1. Why Projects Fail at the Framing Stage
2. The Six Lifecycle Stages
3. Two Parallel Pipelines
4. From Requirement to Learning Task
5. Choosing the Metric Before Training
6. Splitting Data Honestly
7. Human Oversight and Escalation
8. Checkpoint and Summary

Checkpoint Questions

1. Why can a random train/test split give an optimistic result on session-level flow data? Name the mechanism, not just the symptom.
2. Name one feature that would be unavailable at prediction time in a phishing detector, and explain what happens if you leave it in.
3. Which metric would you optimise when a missed intrusion costs far more than a false alarm, and what operational constraint limits how far you can push it?

Three minutes in pairs. Question 1 returns in the midterm and again in Session 8.

In-Class Activity — Pipeline Canvas

Groups of three. Twenty minutes. Each group receives one scenario:

- Lateral movement detection inside a campus network
- Spam and phishing filtering for a university mail service
- Link failure prediction from device telemetry
- Malicious URL blocking at the web proxy

Fill the six canvas boxes: decision, prediction unit, label and its source, features available at prediction time, horizon, error profile. Then add: which split strategy, and who acts on the output. **Report back.** Two groups present. The class critiques the *label definition* first, because that is where most of the disagreement will be.

Summary

- The lifecycle is a cycle with feedback, not a pipeline with an end. Evaluation failures usually send you back to framing or to data.
- Two pipelines run in parallel: data preparation and model development. Data quality caps everything downstream.
- Framing fixes six things: the decision, the prediction unit, the label, the features available at prediction time, the horizon, and the error profile.
- The metric is a consequence of error costs and is frozen before training.
- Security data usually needs a temporal or grouped split; “random 80/20” is a claim to defend.
- Irreversible actions keep a human in the loop, and every decision is logged.

Next session: the tooling that makes all of this reproducible — Colab, NumPy and Pandas on a real flow dataset.

Homework and Reading

Homework (assessed under A1)

- Write a one-page problem definition for a networking or security task of your own choosing, using the six canvas headings plus split strategy
- This document is reused in Session 11 and revisited in Session 15, so choose something you find interesting

Reading

- **[MLS]** Ch. 5, §5.2–5.5 (pp. 275–286), §5.8–5.9 (pp. 293–300), and the chapter summary (p. 304)
- **[Géron]** Ch. 2, “End-to-end machine learning project”: framing and metric selection

Bring your draft canvas to Session 3; we will use it to choose which columns matter.

Slides, notebooks, datasets and submission:
`link.uit.edu.vn/NT120`